Střední průmyslová škola elektrotechnická
Informační technologie
Střední průmyslová škola elektrotechnická, Praha 2, Ječná 30

Miro Face Detector

Rozpoznávání obličeje

Miro Slezák
Information Technology
2026

Contents

1 Project Overview **2**

2 Technology Stack and Justification **2**
2.1 Data Sourcing: Pexels API 2
2.2 Face Detection and Annotation: MediaPipe + OpenCV 2
2.3 Machine Learning Model: Convolutional Neural Network (CNN) 2
2.4 Model Explainability: Grad-CAM 3
2.5 Model Vision: Activation Maximization 3
2.6 Model Architecture 3

3 Architecture and Pipeline **5**
3.1 Phase 1: Data Collection 5
3.2 Phase 2: Preprocessing and Attribute Extraction 5
3.3 Phase 3: Model Training (Google Colab) 5
3.4 Phase 4: Real World Application (Inference) 5

4 Maintainability and Quality Assurance **6**
4.1 Testing and Linting 6
4.2 Configuration and Documentation 6
4.3 Versioning and Workflows 6

5 Data Analysis and Evaluation **8**
5.1 Dataset Distribution 8
5.2 Model Training Performance 8
5.3 Activation Maximization and Filter Grids 9

6 Implementation Plan and Code Separation **10**

1 Project Overview

Goal: Develop a custom machine learning computer vision application capable of detecting a specific person (Miro) in a live camera feed and distinguishing them from other individuals.

Scope: The project strictly focuses on creating a proprietary dataset from scratch to train a binary classification model. The software includes a final user facing application that utilizes the trained model for real time inference.

2 Technology Stack and Justification

2.1 Data Sourcing: Pexels API

The Pexels API provides a generous free tier that allows up to 200 requests per hour. This allows the automated scraping script to download the required negative class images in a single session.

2.2 Face Detection and Annotation: MediaPipe + OpenCV

Developed by Google, MediaPipe provides ultrafast face detection. It is used exclusively in the preprocessing pipeline to find faces in raw images and crop them, ensuring the machine learning model only trains on facial features and ignores backgrounds.

2.3 Machine Learning Model: Convolutional Neural Network (CNN)

A CNN built with TensorFlow and Keras is utilized. CNNs are specifically designed for spatial data like images, making them the optimal choice for learning the visual features that distinguish specific faces.

To demonstrate a clear understanding of the model architecture, the following outlines the core layers used in this network:

- **Convolutional Layer:** This layer applies sliding filters over the 128x128 input images. It acts like a scanner looking for specific patterns, starting with basic edges and progressing to complex facial structures, creating feature maps.
- **Pooling Layer (Max Pooling):** Following a convolution, this layer downsamples the feature maps. It reduces the spatial dimensions while keeping the most important features, which makes the model run more efficiently and helps prevent overfitting.
- **Flatten Layer:** This layer takes the 2D feature maps created by the convolutional layers and unrolls them into a single 1D vector so the data can be read by a standard neural network.
- **Dense (Fully Connected) Layer:** This is the final classification stage. It takes the unrolled spatial features and calculates the final prediction to determine if the face is Miro (1) or Random (0).

2.4 Model Explainability: Grad-CAM

To ensure transparency in the model's decision-making process, the application implements Gradient-weighted Class Activation Mapping (Grad-CAM). This technique uses the gradients of any target concept, flowing into the final convolutional layer to produce a coarse localization map highlighting the important regions in the image for predicting the concept.

In this project, Grad-CAM allows the user to see exactly which facial features (e.g., eyes, nose, jawline) the CNN is using to identify "Miro", providing a layer of interpretability often missing in "black-box" neural networks.

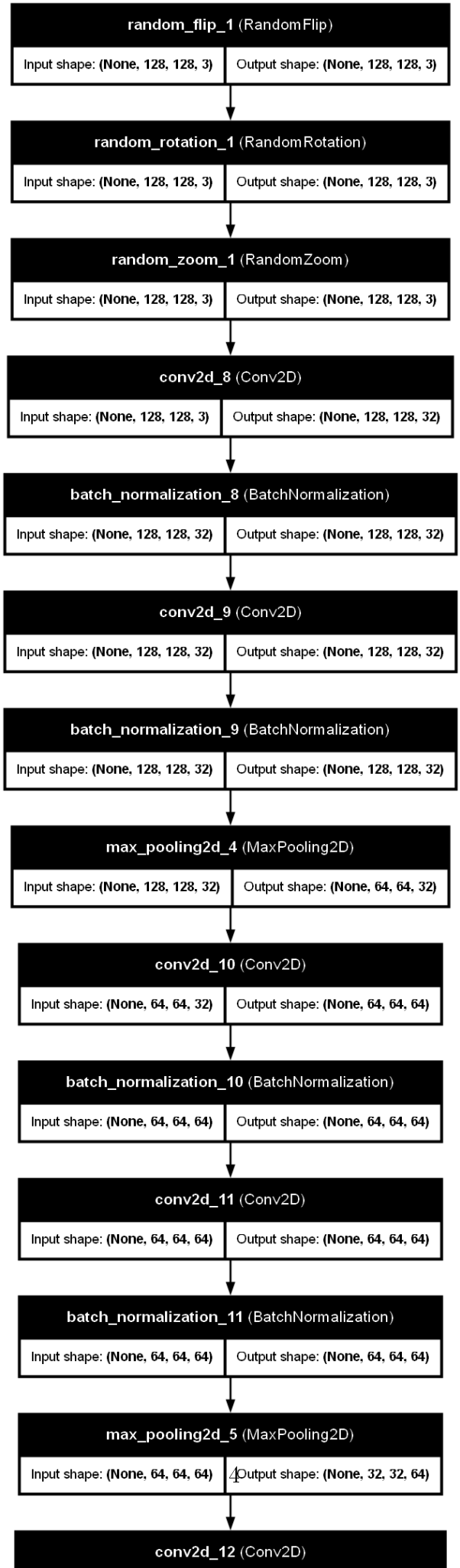
2.5 Model Vision: Activation Maximization

Beyond localizing features in specific images, the project utilizes Activation Maximization to synthesize images that represent the "ideal" input for specific neurons or classes. By performing gradient ascent on a noise image, we can visualize the specific patterns and textures the model has learned to associate with the "Miro" class.

Furthermore, the tool allows for visualizing individual convolutional filters, revealing the hierarchy of features from simple edges in early layers to complex facial structures in deeper layers.

2.6 Model Architecture

The following diagram illustrates the topological structure of the Convolutional Neural Network developed for this project.


random_flip_1 (RandomFlip)
Input shape: (None, 128, 128, 3)
Output shape: (None, 128, 128, 3)
random_rotation_1 (RandomRotation)
Input shape: (None, 128, 128, 3)
Output shape: (None, 128, 128, 3)
random_zoom_1 (RandomZoom)
Input shape: (None, 128, 128, 3)
Output shape: (None, 128, 128, 3)
conv2d_8 (Conv2D)
Input shape: (None, 128, 128, 3)
Output shape: (None, 128, 128, 32)
batch_normalization_8 (BatchNormalization)
Input shape: (None, 128, 128, 32)
Output shape: (None, 128, 128, 32)
conv2d_9 (Conv2D)
Input shape: (None, 128, 128, 32)
Output shape: (None, 128, 128, 32)
batch_normalization_9 (BatchNormalization)
Input shape: (None, 128, 128, 32)
Output shape: (None, 128, 128, 32)
max_pooling2d_4 (MaxPooling2D)
Input shape: (None, 128, 128, 32)
Output shape: (None, 64, 64, 32)
conv2d_10 (Conv2D)
Input shape: (None, 64, 64, 32)
Output shape: (None, 64, 64, 64)
batch_normalization_10 (BatchNormalization)
Input shape: (None, 64, 64, 64)
Output shape: (None, 64, 64, 64)
conv2d_11 (Conv2D)
Input shape: (None, 64, 64, 64)
Output shape: (None, 64, 64, 64)
batch_normalization_11 (BatchNormalization)
Input shape: (None, 64, 64, 64)
Output shape: (None, 64, 64, 64)
max_pooling2d_5 (MaxPooling2D)
Input shape: (None, 64, 64, 64)
Output shape: (None, 32, 32, 64)
conv2d_12 (Conv2D)

3 Architecture and Pipeline

The architecture is divided into a data generation pipeline, a cloud based training phase, and a real time local inference application.

3.1 Phase 1: Data Collection

To ensure the final cleaned dataset meets the strict requirement of at least 1500 records, the collection scripts oversample data.

- **Positive Class (Miro):** A custom script captures frames from multiple videos of the target under varying lighting conditions, generating 1438 initial images saved to the raw data directory. The original unmodified video files are preserved as verifiable proof of non-simulated real data collection.
- **Negative Class (Random):** A script queries the Pexels API for portrait photos and downloads 1200 images into the raw data directory.

3.2 Phase 2: Preprocessing and Attribute Extraction

- All raw images are passed through the MediaPipe Face Detector.
- Images containing zero faces or multiple faces are automatically discarded by the script to ensure data cleanliness.
- Valid faces are cropped using the generated bounding boxes and resized to a strict 128x128 resolution.
- After cropping, the faces are evaluated for sharpness using the Variance of Laplacian method. Any blurry faces below the threshold are automatically excluded from the dataset to prevent false positives.
- **The Data Attributes:** The data loader indexes the images using a `dataset.csv` file. However, the actual training data attributes (parts) utilized by the CNN are the image matrices themselves. Each 128x128x3 RGB image contains 49,152 distinct pixel attributes that the network learns from.
- **Data Splitting:** The final cleaned dataset is split into portions for training and strictly reserved for testing. The split ratio and random state are centrally managed via `config.yaml` (`test_split_size` and `random_seed`) to ensure adaptability.

3.3 Phase 3: Model Training (Google Colab)

The processed `data/` directory and the `dataset.csv` file are uploaded to Google Drive. A Google Colab Jupyter Notebook mounts the drive, loads the preprocessed true data, and trains the CNN. The final trained model weights are exported as a `.keras` file and downloaded back to the local project folder.

3.4 Phase 4: Real World Application (Inference)

The final software is a Python script executable via the command line on the school PC. It requires an external USB webcam connected to the computer, or alternatively, a pre-

recorded video file passed via the `-video` argument. It accesses the video stream using OpenCV, continuously extracts faces, and passes the cropped faces to the trained CNN.

The application draws a bounding box on the live video feed, labeling the face as either "Miro" or "Unknown". The exact confidence threshold for positive classification is customizable via `config.yaml` to ensure adaptability in diverse lighting environments. The user safely terminates the camera feed by pressing the "q" key. When watching a pre-recorded video, the user can also press the Spacebar to pause/resume or use "a" and "d" to skip back and forth.

4 Maintainability and Quality Assurance

To ensure the project is perfectly maintainable, readable, and adheres to strict software engineering standards, the following practices and architectures are implemented.

4.1 Testing and Linting

- **Unit Tests:** The `pytest` framework is used to write unit tests for the data cleaning and transformation pipeline. This verifies that individual functions perform correctly and provides proof of code comprehension.
- **Linting:** `flake8` and `black` are utilized to enforce strict PEP 8 formatting conventions across all authored files in the `src/` directory. This ensures maximum readability.

4.2 Configuration and Documentation

- **Configuration Management:** A `config.yaml` file centralizes all project parameters, such as model hyperparameters, API endpoints, and dataset paths. Sensitive information like API keys are stored in a local `.env` file, which is strictly excluded from version control.
- **README:** A comprehensive `README.md` is provided. It includes step by step instructions for setting up the environment, installing dependencies, and launching the software without an IDE.
- **Project Documentation:** The codebase is thoroughly documented using standard Python docstrings. This generated PDF serves as the primary technical documentation detailing the architecture, data origin, and machine learning pipeline.

4.3 Versioning and Workflows

- **Semantic Versioning:** All releases are tagged using the MAJOR.MINOR.PATCH format (e.g., 1.0.0) to clearly communicate the nature of changes.
- **Changelog:** A `CHANGELOG.md` file is maintained. It categorizes all project updates into Added, Changed, Deprecated, and Removed sections for every version iteration.
- **Automated Workflows (CI/CD):** GitHub Actions are configured to trigger automatically on every pull request. The workflow executes the `pytest` suite and

runs `flake8` linting. If any tests fail or formatting rules are broken, the merge is blocked, guaranteeing code quality.

5 Data Analysis and Evaluation

5.1 Dataset Distribution

Figure 1 shows the distribution of the positive and negative classes in the generated dataset after the cleaning phase.

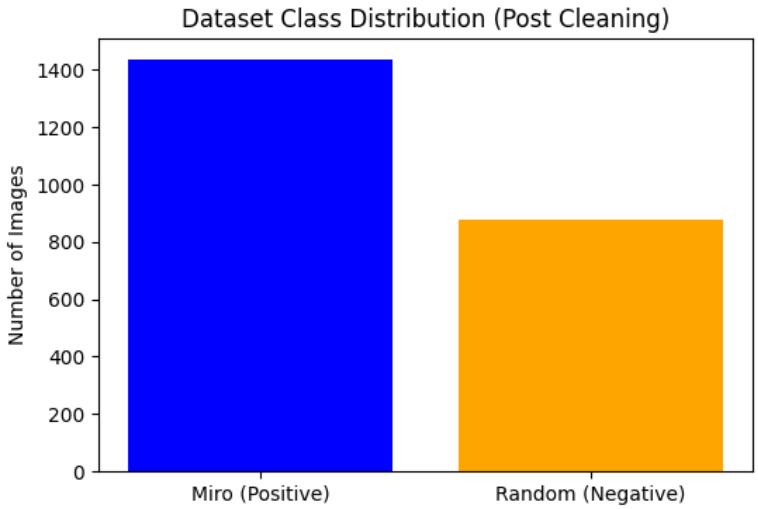


Figure 2: Distribution of positive and negative classes in the cleaned dataset.

5.2 Model Training Performance

Figure 2 illustrates the training and validation accuracy over time.

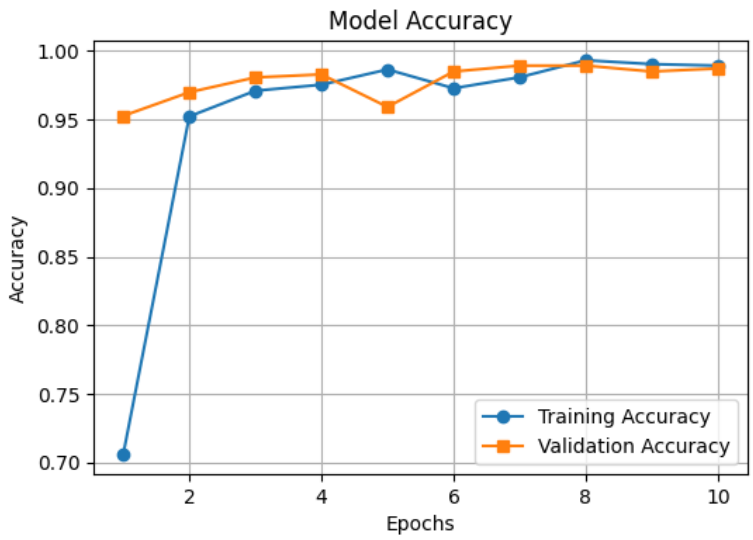


Figure 3: Training and validation accuracy across 10 epochs.

5.3 Activation Maximization and Filter Grids

Figure 3 displays the result of maximizing the output for the "Miro" class, representing the model's internal prototype of the target face. Figure 4 shows a grid of various filters from the final convolutional layer, illustrating the diversity of features the model detects.

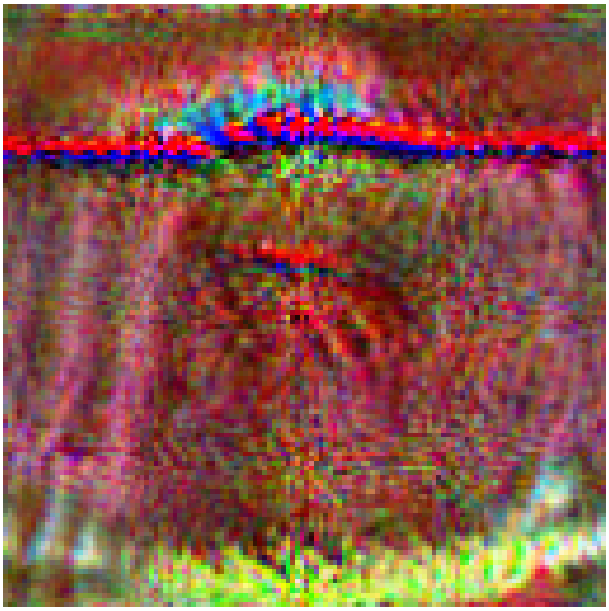


Figure 4: Synthesized image maximizing model activation for the Miro class.

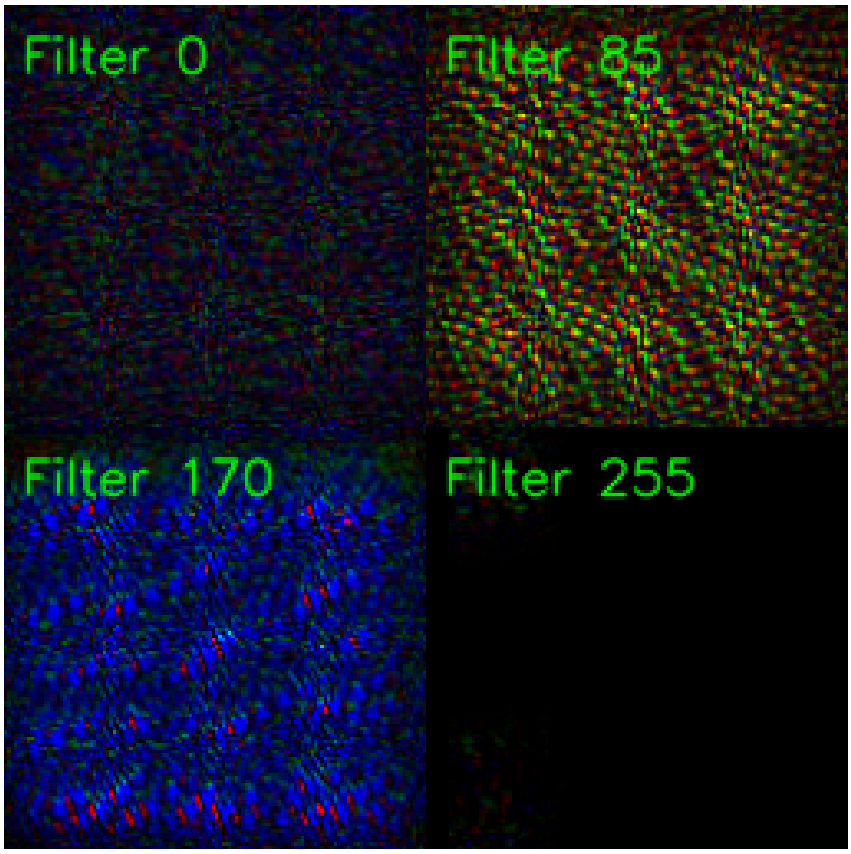


Figure 5: Grid of maximized filters from the convolutional layers.

6 Implementation Plan and Code Separation

To strictly adhere to code authorship requirements, the repository is structured as follows:

1. **src/ (Authored Code)**
   - `pexels_scraper.py`: Custom script to interface with the API and download images.
   - `build_dataset.py`: Custom original pipeline utilizing the MediaPipe library to crop and resize faces.
   - `app.py`: The final webcam application launched by the user.
   - **Rule of Authorship:** Absolutely zero lines of foreign code are present in this directory. All logic is authored by hand.
2. **vendor/ (Foreign Code)**
   - Contains any third party helper libraries, complex boilerplate configurations, snippets explicitly downloaded that was not written by hand. This completely isolates foreign code from the core application logic.
   - `video_extractor.py`: Custom script to extract frames from personal videos. Moved to vendor to separate data collection utilities from core application logic.
3. **data/**
   - `raw/`: Contains the unmodified downloaded images and video frames.
   - `processed/`: Contains the cropped 128x128 faces and the `dataset.csv`.
4. **notebooks/**
   - `model_training.ipynb`: The Jupyter Notebook executed in Google Colab documenting the CNN creation, training, and evaluation.
5. **models/**
   - `miro_detector.keras`: The exported weights of the trained neural network.

Deployment and Execution Instructions: To deploy the application on the target school computer without an IDE, the user must connect an external webcam and execute the following steps in the command line:

1. `python -m venv venv`
2. `venv\Scripts\activate`
3. `pip install -r requirements.txt`
4. `python main.py run` (Use `-video path.mp4` for video files)